

Unit-8: NP-Completeness

Deterministic algorithm and Non- deterministic algorithm

A **deterministic algorithm** is a uniquely defined (determined) sequence of steps for a particular input; that is, given an input and a step during execution of the algorithm, there is only one way to determine the next step that the algorithm can make.

A **non-deterministic** algorithm is an algorithm that can use a special operation that makes a guess when a decision is to be made. A non-deterministic algorithm can provide different outputs for the same input on different executions. Unlike a deterministic algorithm which produces only a single output for the same input even on different runs. The nondeterministic algorithms are often used to find an approximation to a solution, when the exact solution would be too costly to obtain using a deterministic one. A nondeterministic algorithm consists of two phases: **guessing and checking** (verifying). Furthermore, it is assumed that a nondeterministic algorithm always makes a correct guessing.

Polynomial and Super Polynomial Time Complexity

- An algorithm is said to be of **polynomial time complexity** if its running time is upper bounded by a polynomial expression in the size of the input for the algorithm, i.e., $T(n) = O(n^k)$ for some constant k . Polynomial time describes any run time that does not increase faster than n^k , which includes constant time (n^0), logarithmic time ($\log_2 n$), linear time n^1 , quadratic time (n^2) and other higher degree polynomials (like n^3).
- An algorithm is said to take **superpolynomial time** if its running time is not bounded above by any polynomial. For example, an algorithm that runs for 2^n steps on an input of size n requires superpolynomial time (more specifically, exponential time). Superpolynomial time describes any run time that does increase faster than n^k , and includes exponential time (2^n or k^n), factorial time ($n!$), and anything else faster.

Concepts of Tractable and Intractable problems

- The dictionary defines *intractable* as “difficult to treat or work.” This means that a problem in computer science is intractable if a computer has difficulty solving it. This definition is too vague to be of much use to us. To make the notion more concrete, we now introduce the concept of a “polynomial-time algorithm.”

- If the problems can be solved by using **polynomial time** algorithms (i.e. the algorithms whose running time in the worst case is $O(n^k)$ where n is variable and k is constant) then such algorithms are called **tractable problems**. Such problems are considered as efficiently solvable problems (Example: searching, sorting algorithms)
 - We stress that intractability is a property of a problem; it is not a property of any one algorithm for that problem.
 - For a problem to be intractable, there must be no polynomial-time algorithm that solves it. Obtaining a nonpolynomial-time algorithm for a problem does not make it intractable.
- The problems are called **intractable** if they cannot be solved in polynomial time. (Example: Hamiltonian Circuits problem, Halting problem, 0-1 Knapsack problem, TSP)

Problem Reducibility

- A **reduction** is a way to convert one problem to another problem such a way that a solution to the second problem can be used to solve the first problem

Decision Problems and Optimization Problems

- Decision problem D is a problem that has an answer as either “true”, “yes”, “1” or “false”, “no”, “0”. For e.g. if we have the abstract shortest path with instances of the problem and the solution set as $\{0,1\}$, then we can transform that abstract problem by reformulating the problem as “Is there a path from u to v with at most k edges”. In this situation the answer is either yes or no.
- We encounter many problems where there are many feasible solutions and our aim is to find the feasible solution with the best value. This kind of problem is called optimization problem. For e.g. given the graph G , and the vertices u and v find the shortest path from u to v with minimum number of edges. The NP completeness does not directly deal with optimization problems, however we can translate the optimization problem to the decision problem.
- Decision problems are considered less hard than Optimization Problems.
 - Optimization problems are at least harder as Decision problems.
- E.g. the traveling salesperson problem
 - *Optimization version:*
Find the shortest tour

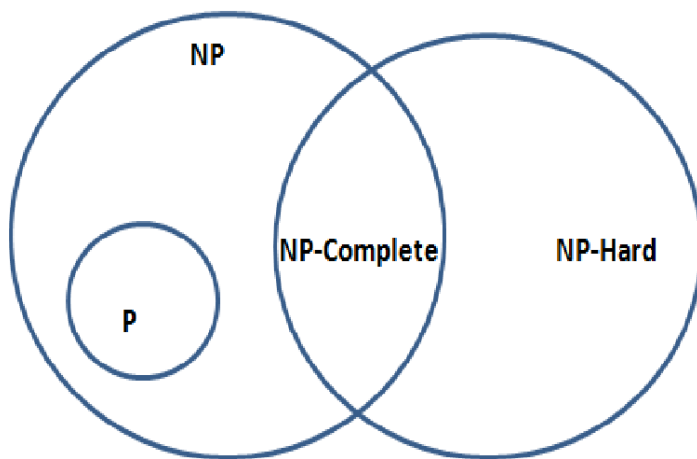
- *Decision version:*

Is there a tour whose total length is less than or equal to a constant C ?

- Decision problems are NP-Complete and Optimization problems are NP- Hard

Complexity classes of problems

Depending on the complexity of algorithms available to solve the problems, there are different classes.



Class P

The set of problems are said to be in class P if they can be solved by deterministic polynomial time algorithms.

- Class P problems are tractable.
- These problems are solvable and verifiable in polynomial time
- Example: sorting, searching, shortest path etc.

Class NP

The set of problems are said to be in class NP ((Nondeterministic Polynomial)) if they can be solved by non-deterministic polynomial time algorithms.

- Class NP problems are non-tractable. (NP problems are also tractable, but only when nondeterministic algorithms are used)
- These problems are not solvable in polynomial time but verifiable in polynomial time

— Example: Sudoku game, Hamiltonian Cycle, etc.

Clearly, $P \subseteq NP$, because deterministic algorithms are those nondeterministic algorithms that do not use nondeterministic decisions. It is also believed that $P \neq NP$; that is, there exist problems with nondeterministic polynomial algorithms that cannot be solved with deterministic polynomial algorithms. (Is $P = NP$? -- million dollar problem)

NP –Complete

A problem is called **NP-complete** if it is NP (it can be solved efficiently by a nondeterministic polynomial algorithm) and every NP problem can be polynomially reduced to this problem.

- A problem X is NP-Complete, if it is in NP and there is any NP problem Y, such that Y is reducible to X in polynomial time.
- Because reducibility is a transitive relation, we can also say that an NP problem P1 is NP-complete if there is an NP-complete problem P2 that is polynomially reducible to P1.
- All NP-complete problems are computationally equivalent; that is, if an NP-complete problem can be solved with a deterministic polynomial algorithm, then so can be all NP-complete problems, and thus $P = NP$. Also, if any problem in NP is intractable, then all NP-complete problems are intractable.
- Example: SAT (Satisfiability) problem, Vertex cover problem

NP –Hard

- These are the problems that are at least as hard as the hardest problem in NP.
Note that *NP-hard problems do not have to be in NP*.
- If we can solve these problems in polynomial time, we can solve any NP problem that can possibly exist.
- A problem X is NP-Hard, if there is an NP problem Y, such that Y is reducible to X in polynomial time.
- Example: Halting Problem, TSP (Travelling Salesman Problem)
- *All NP – Complete problems are NP-hard, but all NP – Hard problems are not NP-Complete problems.*

NOTES:

- It is known $P \subseteq NP$.
- However, it is not known whether $P=NP$ or whether P is a proper subset of NP
- It is believed NP is much larger than P
 - We cannot find a polynomial-time algorithm for many NP problems.
 - But, no NP problem is proved to have exponential lower bound. (No NP problem has been proved to be not in P .)
- So, “does $P = NP$?” is still an open question!
- Decision problems are NP -Complete and Optimization problems are NP -Hard
- NP -Complete problems are the “hardest” problems in NP
- All NP – Complete problems are NP -hard, but all NP – Hard problems are not NP - Complete problems.
- If any *one* NP -Complete problem can be solved in polynomial time...
 - ...then *every* NP -Complete problem can be solved in polynomial time...
 - ...and in fact *every* problem in **NP** can be solved in polynomial time (which would show **$P = NP$**)

Cook’s Theorem

Also known as Cook–Levin theorem

- Theorem : “Boolean Satisfiability is NP -complete”
- Proved in Cook Levin Theorem (Cook’s Theorem) :
 - SAT is in NP .
 - All problems in NP are polynomially reducible to SAT.

SAT Problem

- Also Called Boolean Satisfiability or simple Satisfiability problem.
- Definition (SAT Problem) :

Given a Boolean expression, does it have a satisfying truth assignment?

Example

The expression $\neg((x_1 \vee \neg(x_2 \wedge (x_3 \implies x_2))) \vee \neg x_3)$ is SAT.

There is a satisfying truth assignment $x_1 = 0, x_2 = 1, x_3 = 1$.

$x_1 \wedge \neg x_1$ is UNSAT. There is no satisfying truth assignment.

- SAT was the first known **NP-complete** problem.

CNF-SAT

- ▶ *Literal*: Boolean variable or its negation (e.g. x or $\neg x$).
- ▶ *Clause*: Logical OR of one or more literals (e.g. $x_1 \vee x_2 \vee \neg x_3$).
- ▶ A boolean expression is in *CNF* if it's the logical AND of clauses (e.g. $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (x_4) \wedge (\neg x_2 \vee \neg x_3 \vee x_4)$).

- CNF-SAT is a NP-Complete problem

Variants of CNF-SAT

- ▶ **k-SAT** = k literals in each clause.
 - ▶ k-SAT is NP-complete.
- ▶ **3-SAT** = 3 literals in each clause.
 - ▶ e.g. $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_4)$
 - ▶ 3-SAT is NP-complete.

Circuit satisfiability problem

- Circuit satisfiability problem (Circuit SAT) is the question “Given a Boolean combinational circuit, is it satisfiable? i.e. does the circuit has assignment sequence of truth values that produces the output of the circuit as 1?” Given the circuit satisfiability problem take a circuit x and a certificate y with the set of values that produce output 1, we can verify that whether the given certificate satisfies the circuit in polynomial time. So we can say that circuit satisfiability problem is NP.

NP Completeness and Problem Reducibility

- A **reduction** is a way to convert one problem to another problem such a way that a solution to the second problem can be used to solve the first problem

We want prove some problems are computationally difficult.

- As a first step, we settle for relative judgements:

Problem X is at least as hard as problem Y

- To prove such a statement, we reduce problem Y to problem X:

If you had a black box that can solve instances of problem X, how can you solve any instance of Y using polynomial number of steps, plus a polynomial number of calls to the black box that solves X ?

- Polynomial reduction can be used for polynomial time solution
- If **A** polynomially reduces to **B**, and we know a polynomial algorithm to **B**, then we can solve **A** in polynomial time as well

We reduce to the problem we want to show is the harder problem.

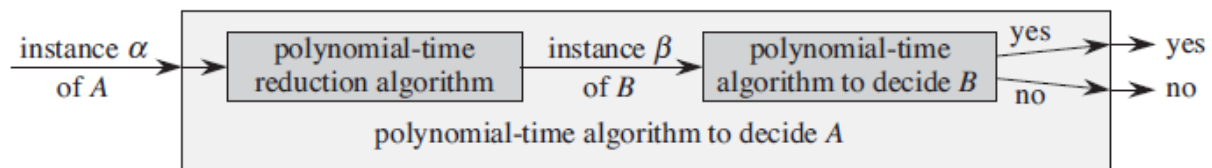
- We can prove a new problem is NP-complete by reducing some other NP-complete problem to it.

The polynomial reduction is a transitive relation.

- If **A** polynomially reduces to **B** and **B** polynomially reduces to **C** then this means, **A** polynomially reduces to **C**

- The crux of NP-Completeness is *reducibility*
 - Informally, a problem P can be reduced to another problem Q if *any* instance of P can be “easily rephrased” as an instance of Q, the solution to which provides a solution to the instance of P
 - *What do you suppose “easily” means? ->Polynomial Time*
 - This rephrasing is called *transformation*
 - Intuitively: If P reduces to Q, P is “no harder to solve” than Q

- Let us consider a decision problem A, which we would like to solve in polynomial time. We call the input to a particular problem an **instance** of that problem. Now suppose that we already know how to solve a different decision problem B in polynomial time. Finally, suppose that we have a procedure that transforms any instance α of A into some instance β of B with the following characteristics:
 - The transformation takes polynomial time.
 - The answers are the same. That is, the answer for α is “yes” if and only if the answer for β is also “yes.”



We call such a procedure a **polynomial-time reduction algorithm**. As above figure shows, it provides us a way to solve problem A in polynomial time:

1. Given an instance α of problem A, use a polynomial-time reduction algorithm to transform it to an instance β of problem B.
2. Run the polynomial-time decision algorithm for B on the instance β .
3. Use the answer for β as the answer for α .

As long as each of these steps takes polynomial time, all three together do also, and so we have a way to decide on α in polynomial time. In other words, by “reducing” solving problem A to solving problem B, we use the “easiness” of B to prove the “easiness” of A.

- An example:
 - A: Given a set of Booleans, is at least one TRUE?
 - B: Given a set of integers, is their sum > 0 ?
 - Transformation: Given $(x_1, x_2, \dots, x_n)$ construct $(y_1, y_2, \dots, y_n)$ where $y_i = 1$ if $x_i = \text{TRUE}$, $y_i = 0$ if $x_i = \text{FALSE}$
- Another example:
 - Solving linear equations is reducible to solving quadratic equations
 - How can we easily use a quadratic-equation solver to solve linear equations?

How to prove some problem is NP Complete?

- NP-completeness is about showing how hard a problem is rather than how easy it is, we use polynomial-time reductions in the opposite way to show that a problem is NP-complete.
- Let us take the idea a step further, and show how we could use **polynomial-time reductions to show that no polynomial-time algorithm can exist for a particular problem B**. Suppose we have a **decision problem A for which we already know that no polynomial-time algorithm can exist**. (Let us not concern ourselves for now with how to find such a problem A.)
- Suppose further that we **have a polynomial-time reduction transforming instances of A to instances of B**. Now we can use a simple **proof by contradiction** to show that no polynomialtime algorithm can exist for B. Suppose otherwise; i.e., suppose that B has a polynomial-time algorithm. Then, using the method shown in above figure would have a way to solve problem A in polynomial time, which contradicts our assumption that there is no polynomial-time algorithm for A.
- For NP-completeness, we cannot assume that there is absolutely no polynomial time algorithm for problem A. The proof methodology is similar, however, in that **we prove that problem B is NP-complete on the assumption that problem A is also NP-complete**.

Steps to follow in order to prove some problem X is NP-Complete

- Show that the problem X is in NP (i.e Show $X \in \text{NP}$)
- Choose a known NP- Complete problem Y
- Show that the problem Y is polynomially reducible to X (i.e. Show $Y \leq_p X$)

Theorem: 3 CNF SAT (3 SAT) is NP – Complete

Proof:

- 3 SAT is a satisfiability problem for a Boolean formula which is in conjunctive normal form (CNF) and each clause has exactly 3 literals.

Example: $(x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee \neg x_7 \vee x_5) \wedge (x_6 \vee x_8 \vee \neg x_9)$

- **3SAT problem is in NP** since any assignment of values to variables can be verified in polynomial time. So, **3SAT $\in$ NP**.
- Let us pick **SAT problem as known NP-Complete problem**.
- Now we have to show SAT is polynomially reducible to 3SAT (**SAT $\leq_p$ 3SAT**)
- Let us take a formula F of SAT. We transform it into a formula F' of 3SAT such that F' is satisfiable if and only if F is satisfiable.

— Each clause of F is transformed into a sub-expression of F'.

- Clauses of length 3 are left unchanged.
- A clause of length 1, such as (x) is changed as follows

$$(x \vee y_1 \vee y_2) \wedge (x \vee \neg y_1 \vee \neg y_2) \wedge (x \vee \neg y_1 \vee \neg y_2)$$

where y_1 and y_2 are two new variables added specifically for the transformation of that clause.

- A clause of length 2, such as $(x_1 \vee x_2)$ is changed as follows

$$(x_1 \vee x_2 \vee y) \wedge (x_1 \vee x_2 \vee \neg y)$$

where y is a new variable added specifically for the transformation of that clause.

- For a clause of length $k > 3$, such as $(x_1 \vee \dots \vee x_k)$, we change it as follows

$$(x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_4 \vee y_3) \wedge \dots \wedge (\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

where $y_1, \dots, y_{k-3}$ are new variables added specifically for the transformation of that clause.

- The **above reduction can be carried out in polynomial time**.
- Hence **3SAT problem is NP-Complete**

Assignment:

- Prove Vertex cover is NP-Complete**
- Prove Sub-Set Problem is NP-Complete**

Approximation Algorithms

Many problems of practical significance are NP-complete, yet they are too important to abandon merely because we don't know how to find an optimal solution in polynomial time. Even if a problem is NP-complete, there may be hope. We have **at least three ways to get around NP-completeness**.

- First, **if the actual inputs are small**, an algorithm with exponential running time may be perfectly satisfactory.
- Second, we may be able to **isolate important special cases that we can solve in polynomial time**.
- Third, we might come up with **approaches to find near-optimal solutions in polynomial time** (either in the worst case or the expected case).

In practice, near optimality is often good enough. We call an algorithm that returns near-optimal solutions an **approximation algorithm**

- An approximate algorithm is a way of dealing with NP-completeness for optimization problem.
- This technique does not guarantee the best solution. The goal of an approximation algorithm is to come as close as possible to the optimum value in a reasonable amount of time which is at most polynomial time.
- Approximation algorithm returns near-optimal solution.
- If we are dealing with optimization problem (maximization or minimization) with feasible solution having positive cost then it is worthy to look at approximate algorithm for near optimal solution.
- An algorithm has an **approximation ratio** of $\rho(n)$ –if, for any problem of input size n , the cost C of solution by an algorithm and the cost C^* of optimal solution have the relation as $\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n)$
- If an algorithm achieves an approximation ratio of $\rho(n)$, we call it **$\rho(n)$ approximation algorithm**.
- The definitions of the approximation ratio and of a $\rho(n)$ - approximation algorithm apply to both **minimization and maximization problems**.

- For a maximization problem, $0 < C \leq C^*$ and the ratio $\frac{C^*}{C}$ gives the factor by which the cost of an optimal solution is larger than the cost of the approximate solution. Similarly, for a minimization problem, $0 < C^* \leq C$, and the ratio $\frac{C}{C^*}$ gives the factor by which the cost of the approximate solution is larger than the cost of an optimal solution.
- Approximation ratio of $\rho(n)$, is always greater than or equal to 1. If solution produced by approximation algorithm is true optimal solution then clearly we have $\rho(n) = 1$
- Therefore, a 1-approximation algorithm produces an optimal solution, and an approximation algorithm with a large approximation ratio may return a solution that is much worse than optimal.

NOTE: When the approximation ratio is independent of n , we use the terms “approximation ratio of ρ ” and “ ρ - approximation algorithm,” indicating no dependence on n .

Approximation Algorithm for Vertex Cover Problem

- A **vertex cover** of an undirected graph $G(V, E)$ is a subset $V' \subseteq V$ such that if (u, v) is an edge of G , then either $u \in V'$ or $v \in V'$ (or both).
- The size of a vertex cover is the number of vertices in it.
- The **vertex-cover problem** is to find a vertex cover of minimum size in a given undirected graph. We call such a vertex cover an *optimal vertex cover*.
- This problem is the optimization version of an NP-complete decision problem. Even though we don't know how to find an optimal vertex cover in a graph G in polynomial time, we can efficiently find a vertex cover that is near-optimal.
- The following approximation algorithm takes as input an undirected graph G and returns a vertex cover whose size is guaranteed to be *no more than twice the size of an optimal vertex cover*.

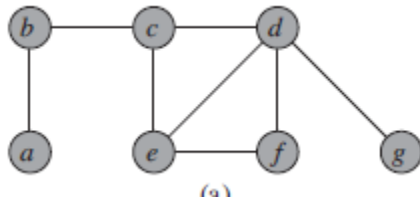
APPROX-VERTEX-COVER(G)

```

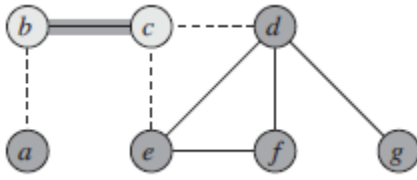
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```

Example

Given a graph $G(V,E)$ as



- Initially $C = \{\}$, $E' = \{(a,b), (b,c), (c,d), (c,e), (d,e), (e,f), (d,f), (d,g)\}$
- Take an arbitrary edge (b,c) The edge (b,c) shown heavy, is the first edge chosen. Vertices b and c , shown lightly shaded, are added to the set C containing the vertex cover being created. Edges (a,b) and (c,d) , shown dashed, are removed since they are now covered by some vertex in C .

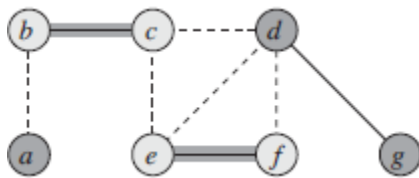


Now,

$$C = \{b, c\}$$

$$E' = \{(d,e), (e,f), (d,f), (d,g)\}$$

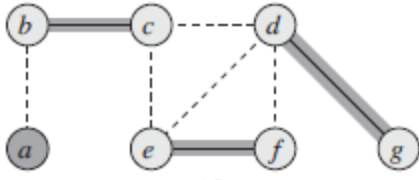
- Now, edge, (e,f) is chosen; vertices e and f are added to C .



$$C = \{b, c, e, f\}$$

$$E' = \{(d,f), (d,g)\}$$

- Next, Edge (d,g) is chosen; vertices d and g are added to C .



$C = \{b, c, e, f, d, g\}$

$E' = \{\}$

- Here, C contains 6 vertices as vertex cover.
- Though this is not optimal solution it is the result of approximation algorithm.
- The optimal solution for this problem?
 - $C = \{b, d, e\}$ with vertex cover size 3

NOTE: Solutions can be different based on choices in different steps.

Analysis

- If E' is represented using the adjacency lists the above algorithm takes $O(V+E)$ since each edge is processed only once and every vertex is processed only once throughout the whole operation.

Theorem: APPROX-VERTEX-COVER is a polynomial-time 2-approximation algorithm (Refer book for proof)

Assignment:

Write the Approximation Algorithm for Sub-Set Sum Problem and analyze the time complexity. Explain the algorithm with suitable example.